Estructura de Datos

Tema 8:
Árboles

GRADO EN INGENIERÍA DEL SOFTWARE

Salvador Sánchez y Cristian Rodríguez

Universidad
Rey Juan Carlos

Curso 2023-2024

Contenidos

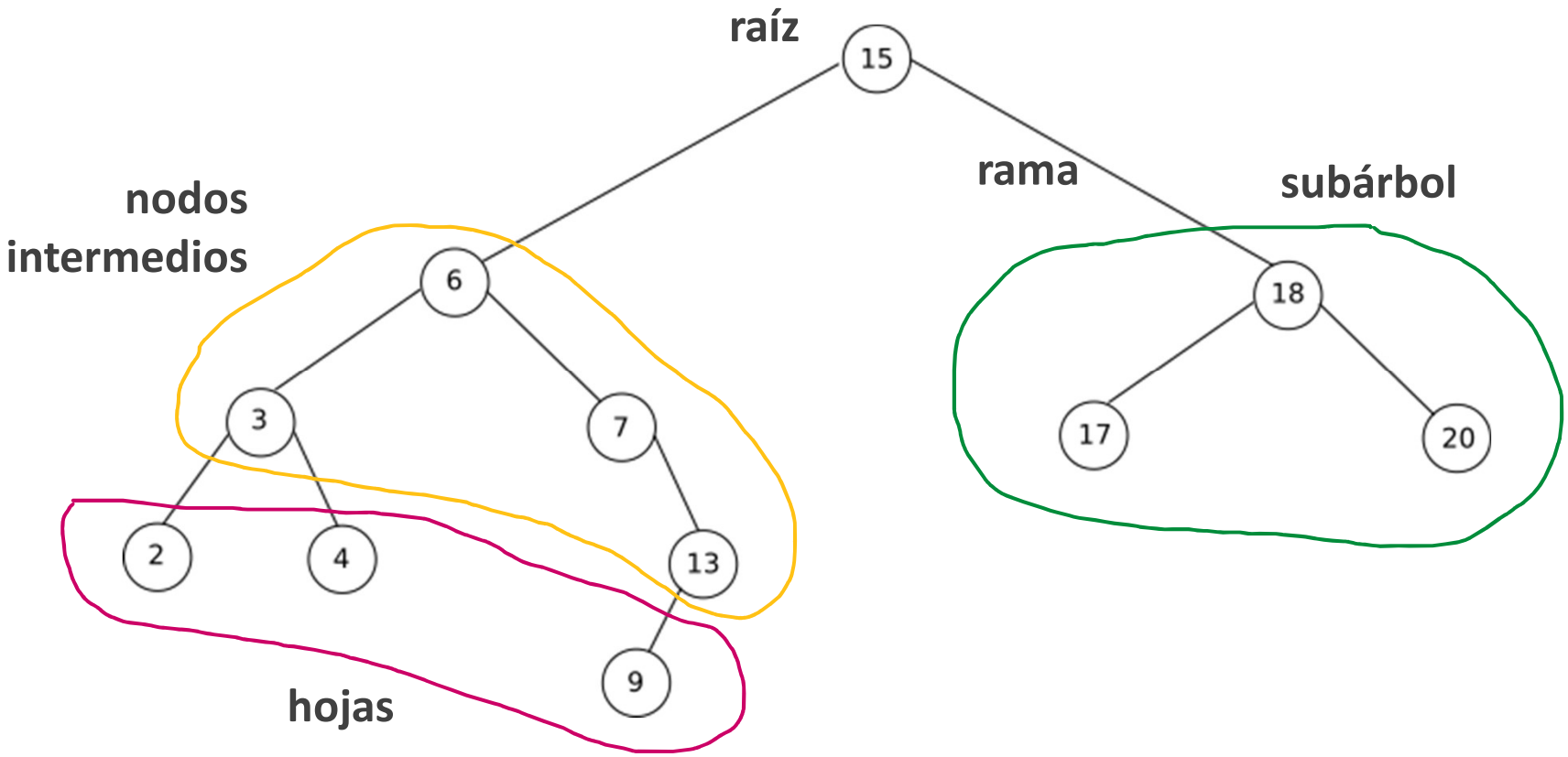
- Definiciones
- Tipos de árboles
  - Árboles generales, binarios y binarios de búsqueda
  - Recorridos
  - Especificaciones
  - Implementación
- Genericidad
- Árboles AVL
- Ejercicios propuestos

Definiciones

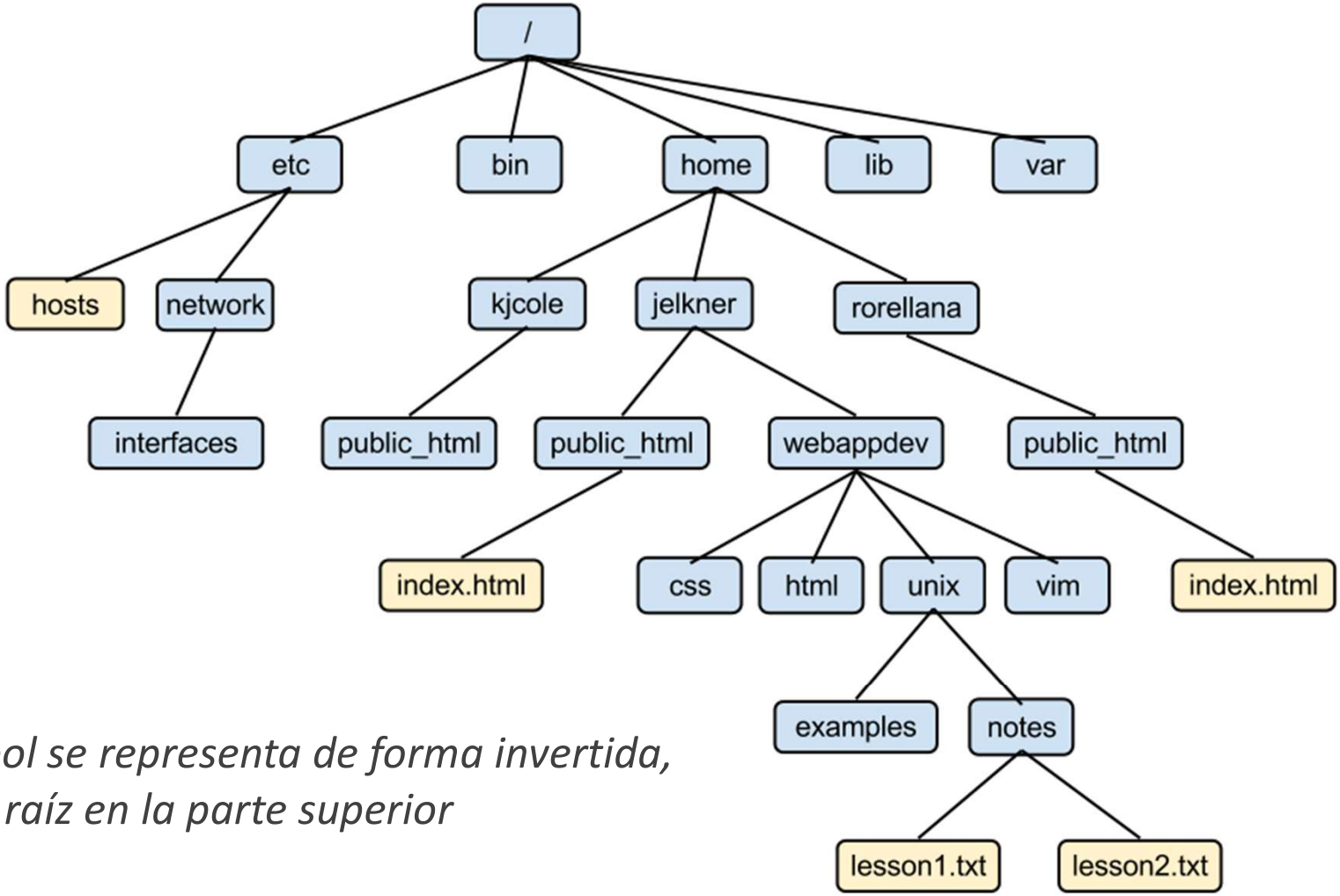
- Un **árbol** es una estructura de datos abstracta, no lineal, cuyos elementos (nodos) se almacenan jerárquicamente
  - …¡Y recursiva! → cualquier subárbol es a su vez un árbol

- Todo nodo tiene un ascendiente (padre) y –potencialmente– varios descendientes (hijos)
  - Nodo "raíz": inicio de la jerarquía
  - Nodos "hoja" o terminales: sin descendientes
  - Rama: conexión entre nodos

Árboles: elementos



Árboles: representación



Un árbol se representa de forma invertida, con su raíz en la parte superior

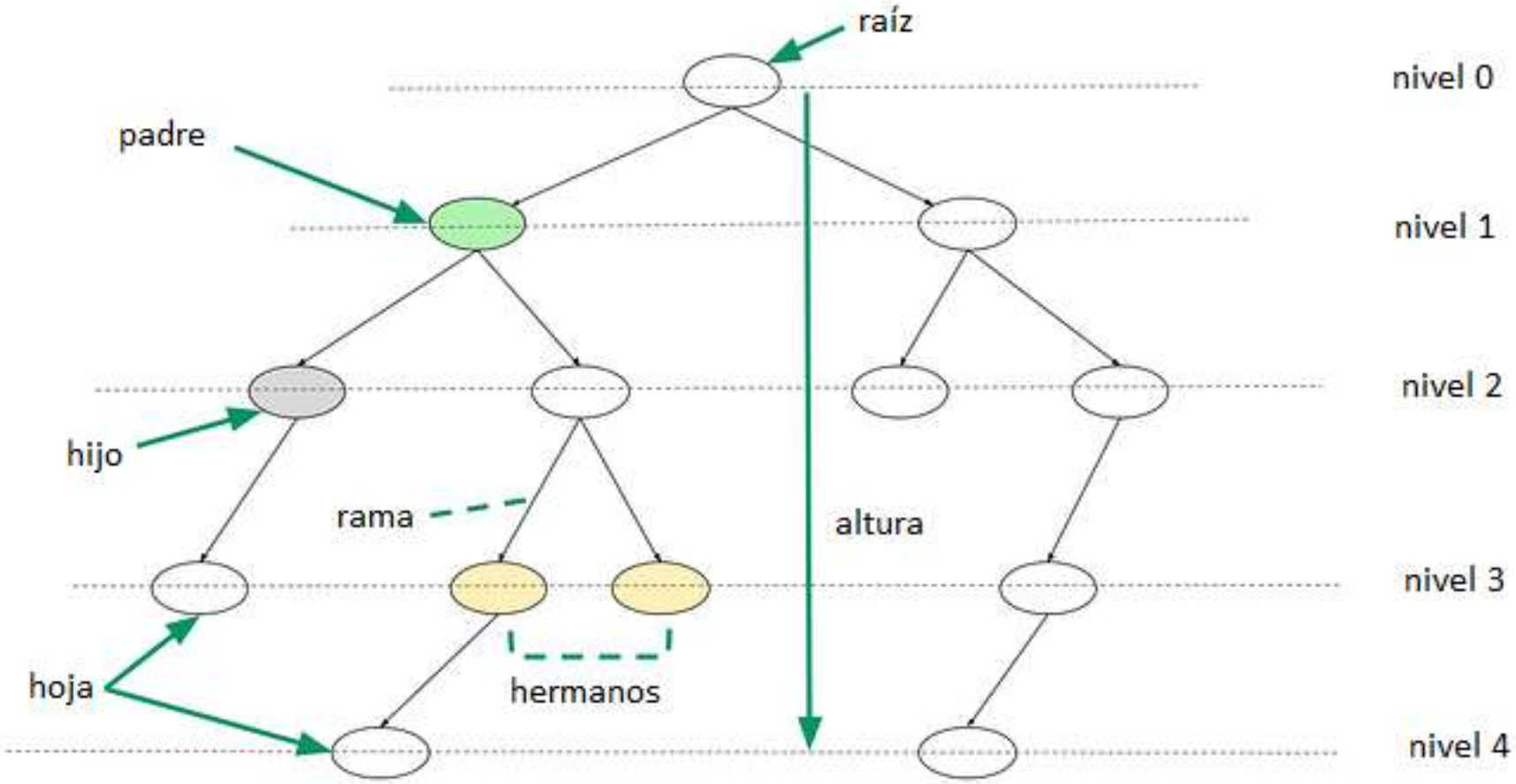
Descendientes, caminos y niveles

- El máximo número de **descendientes** puede ser fijo o no:
  - Un árbol n-ario tiene máximo n hijos por nodo
  - Cada nodo está conectado por una única rama a su nodo padre
- **Camino**: secuencia de nodos entre dos nodos
  - Su longitud es el número de ramas entre ellos
- **Nivel** de un nodo: distancia al nodo raíz
  - El nodo raíz está en el nivel 0

Grado, altura, aridad

- **Grado de un nodo**: número de hijos
- **Grado o aridad de un árbol**: máximo grado de sus nodos
- **Altura** o profundidad de un árbol es el nivel de la hoja del camino más largo desde la raíz más uno.
  - La altura de un árbol vacío es 0

Profundidad y niveles en un árbol

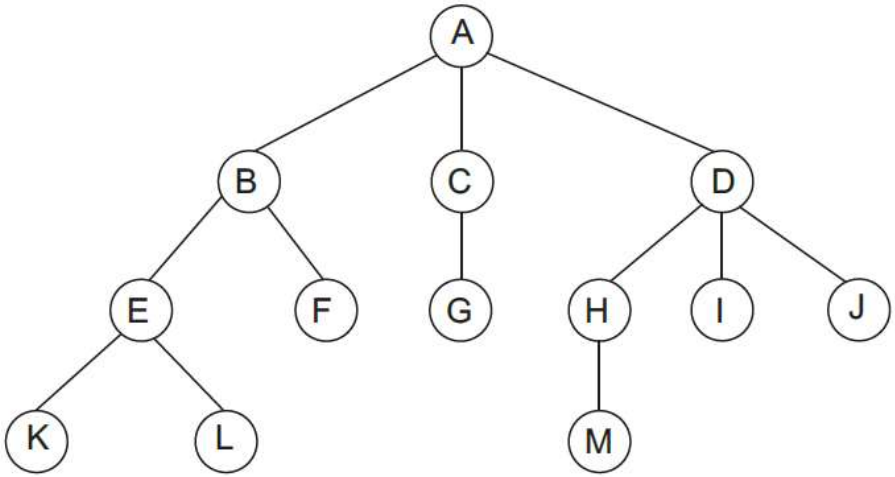


Tipos de árboles

Árboles generales

Árboles generales

- Un árbol general consta de nodos y ramas que parten de un nodo inicial o raíz
- No existen un límite en el número de hijos de un nodo
- Un árbol general puede ser vacío o tener una raíz y más nodos y ramas.



Árboles generales

- Para implementar un árbol general se utiliza:
  - Un manejador que, en Pascal, frecuentemente es un puntero al nodo raíz
  - Una estructura nodo con información y enlaces a los hijos

```
tarbolgeneral = ^tnodo;
```

```
tnodo = RECORD
  info: tipo_base;
  hijos: tlistaNodos;
end;
```

(a) lista de hijos

```
tnodo = RECORD
  info: tipo_base;
  firstChild, nextSibling: ^tnodo;
end;
```

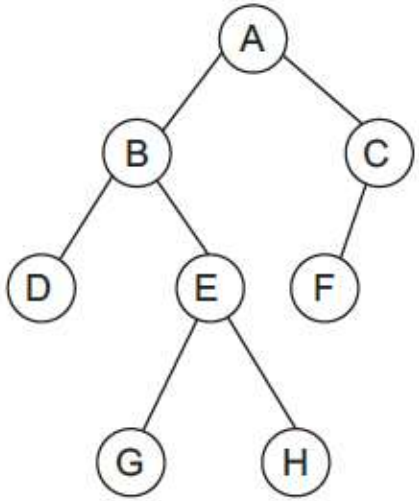
(b) hijo izquierdo y hermanos

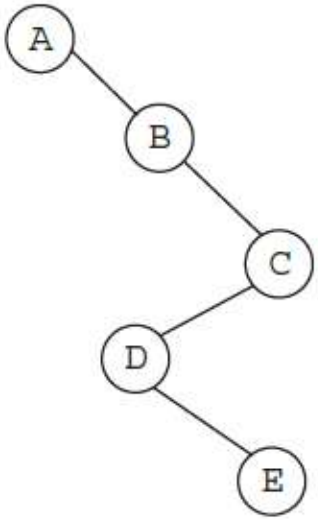
Tipos de árboles

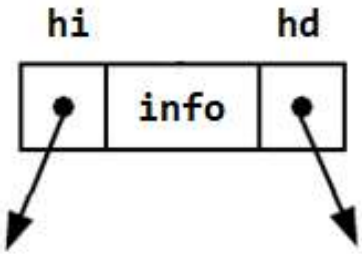
Árboles binarios

Árboles binarios

- Árboles cuyos nodos pueden tener a lo sumo dos hijos
- Los nodos tienen información + referencias a los hijos





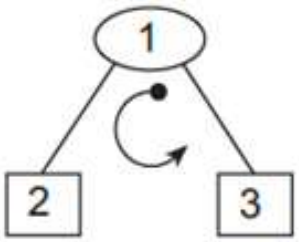


Árboles binarios

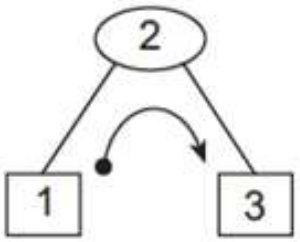
- A partir del nodo raíz de un árbol se puede acceder a los demás nodos del árbol: de ahí el manejador al nodo raíz.
- Los nodos hijo izquierdo y derecho de cada nodo son a su vez la raíz de otros sub-árboles binarios, y así recursivamente hasta llegar a las hojas.

Recorridos

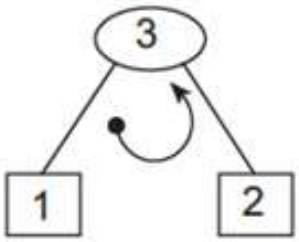
- El recorrido de un árbol binario requiere que cada nodo del árbol sea visitado solo una vez en una secuencia predeterminada.
  - Recorrido **en profundidad**: preorden, post-orden e in-orden
  - Recorrido **en anchura**: horizontalmente desde la raíz a todos sus hijos, a continuación, a los hijos de sus hijos y así sucesivamente.



Recorrido *preorden*



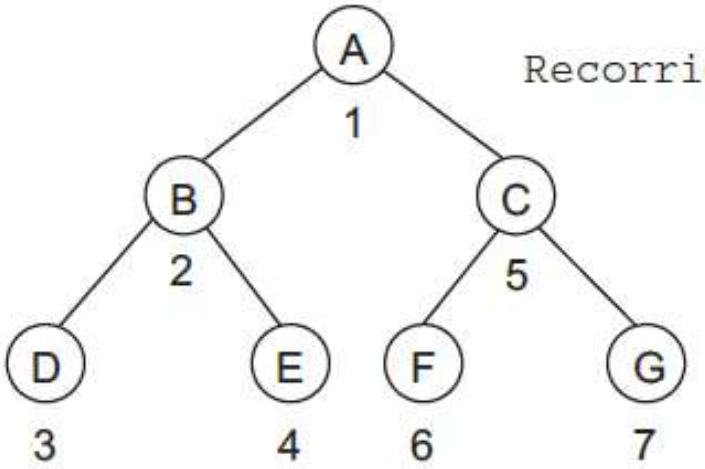
Recorrido *enorden*



Recorrido *postorden*

pre-orden

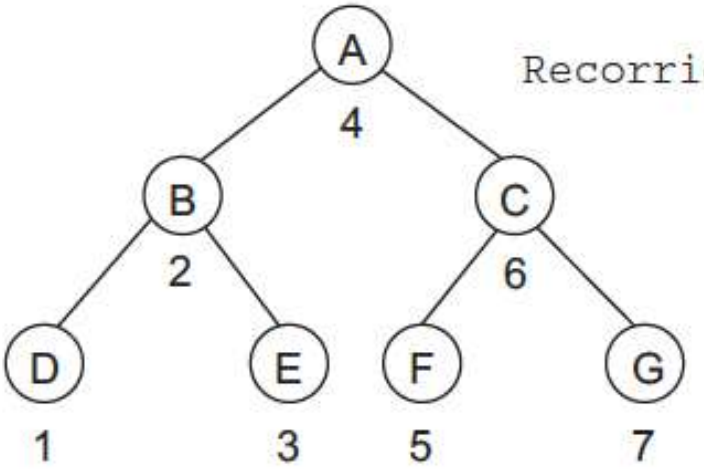
- Un recorrido en preorden visita el nodo raíz antes que los hijos:
  1. Visitar el nodo raíz.
  2. Recorrer el subárbol izquierdo en preorden.
  3. Recorrer el subárbol derecho en preorden.



Recorrido: A, B, D, E, C, F, G

in-orden

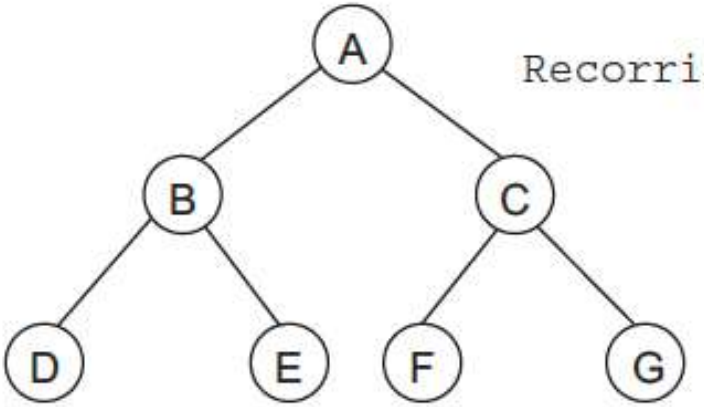
- Un recorrido en in-orden visita el subárbol izquierdo, luego el nodo raíz y finalmente el subárbol derecho:
  1. Recorrer el subárbol izquierdo en in-orden
  2. Visitar el nodo raíz
  3. Recorrer el subárbol derecho en in-orden.



Recorrido: D, B, E, A, F, C, G

post-orden

- Un recorrido en post-orden visita el subárbol izquierdo, luego el subárbol derecho y finalmente el nodo raíz :
  1. Recorrer el subárbol izquierdo en post-orden
  2. Recorrer el subárbol derecho en post-orden.
  3. Visitar el nodo raíz



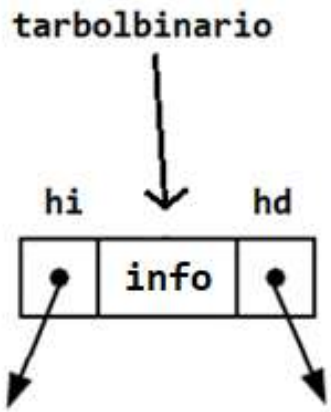
Recorrido: D-E-B-F-G-C-A

TAD Árbol binario

- **Nombre:** Árbol binario.
- **Valores posibles:** *vacío, otros*.
- **Operaciones:**
  - `add`
  - `remove`
  - `is_empty`
  - `in`
  - `initialize`
- **Uso de otros tipos:** tipo base, arrays / punteros (según implementación)

Implementación

```
interface
  type
    tbinaryTree = ^tnode;
    tnode = RECORD
      info: integer;
      hi, hd : tbinaryTree;
    end;

    type

  // Basic methods
  procedure initialize(var a:tbinaryTree);
  function is_empty(a:tbinaryTree): boolean;
  procedure add(var a:tbinaryTree; elem: integer);
  function in_tree(a:tbinaryTree; elem: integer): boolean;
  procedure remove(var a:tbinaryTree; elem: integer);

  // Traversal algorithms
  procedure preorder(a: tbinaryTree);
  procedure inorder(a: tbinaryTree);
  procedure postorder(a: tbinaryTree);

  // Other methods
  function node_count(a:tbinaryTree): integer;
  function get_hi(a:tbinaryTree): tbinaryTree;
  function get_hd(a:tbinaryTree): tbinaryTree;
```

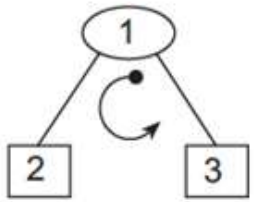


Implementación

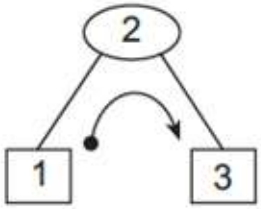
```
procedure initialize(var a:tbinaryTree);
begin
  a:= nil;
end;

function is_empty(a:tbinaryTree): boolean;
begin
  is_empty:= a=nil;
end;

function in_tree(a: tbinaryTree; elem : integer) : boolean;
begin
  if a = NIL then in_tree := FALSE
  else if a^.info = elem then in_tree := TRUE
  else in_tree := (in_tree(a^.hd, elem)) or (in_tree(a^.hi, elem))
end;
```

Implementación

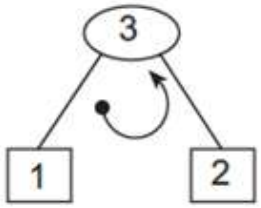
```
procedure preorder(a: tbinaryTree);
begin
  if (a <> NIL) then begin
    visit(a^.info);
    preorder(a^.hi);
    preorder(a^.hd)
  end
end;

procedure inorder(a: tbinaryTree);
begin
  if (a <> NIL) then begin
    inorder(a^.hi);
    visit(a^.info);
    inorder(a^.hd)
  end
end;

procedure postorder(a: tbinaryTree);
begin
  if (a <> NIL) then begin
    postorder(a^.hi);
    postorder(a^.hd);
    visit(a^.info);
  end
end;
```



preorden



enorden



postorden

Implementación

```
function get_hi(a:tbinaryTree): tbinaryTree;
begin
  if a = nil then get_hi := nil
  else get_hi:= a^.hi;
end;

function get_hd(a:tbinaryTree): tbinaryTree;
begin
  if a = nil then get_hd := nil
  else get_hd:= a^.hd;
end;
```

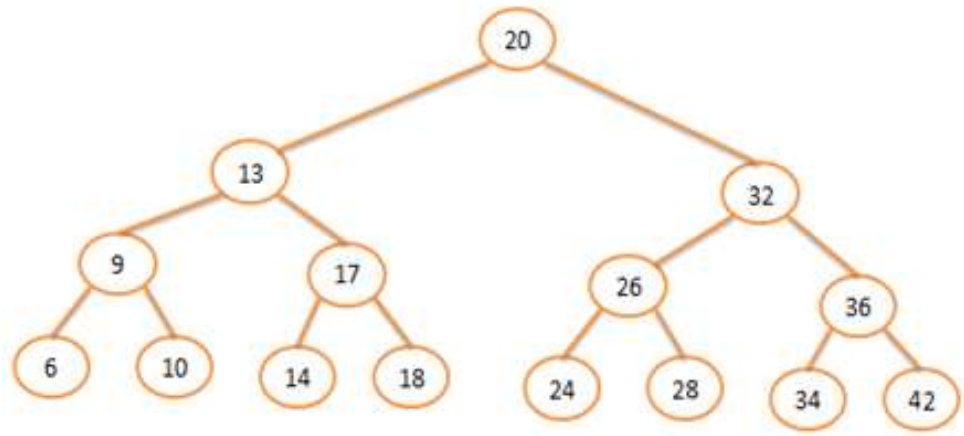
Los métodos add y remove dependen de reglas que puedan imponerse a cada tipo de árbol, por ejemplo, si el árbol está nivelado, ordenado, etc. → No se implementarán aquí

Tipos de árboles

Árboles binarios de búsqueda

Árbol binario de búsqueda

- En un árbol binario de búsqueda (BST, *Binary Search Tree*)…
  - Todos los datos del subárbol izquierdo de un nodo son menores que los datos de ese nodo
  - Todos los datos del subárbol derecho son mayores.
- Se puede buscar en ellos un término utilizando un algoritmo de **búsqueda binaria** similar al empleado en los arrays

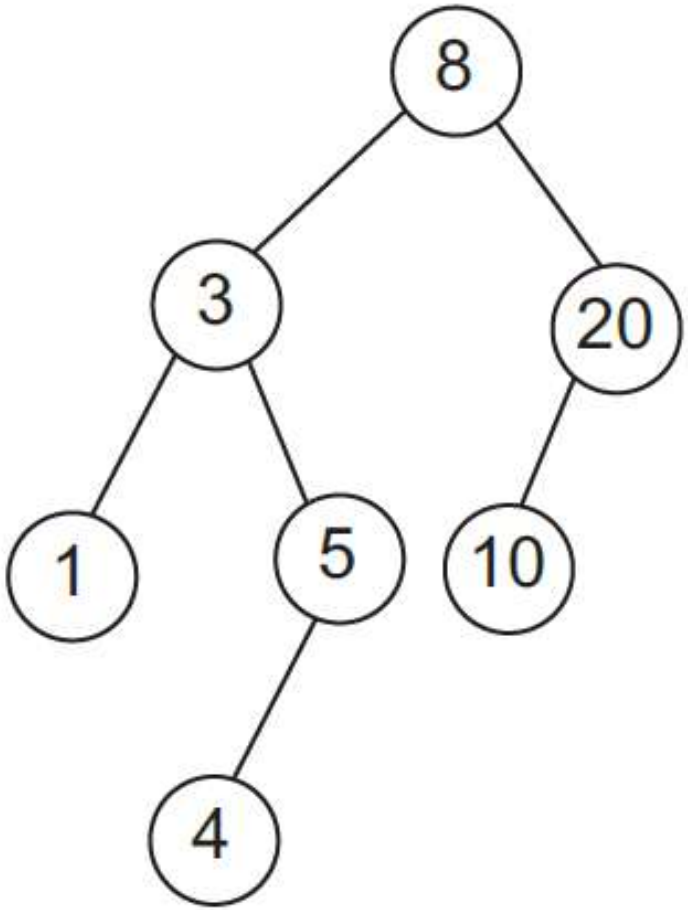


Árbol binario de búsqueda

- Estructura recursiva: cada subárbol es a su vez un árbol binario de búsqueda
- Debe haber definida una función “menor que” que permita mantener los datos ordenados en la jerarquía del árbol
- Es posible realizar los mismos recorridos que en un árbol binario, lo único que cambia es la inserción y borrado

Inserción de un elemento

- Cada nuevo elemento se inserta como una *hoja* en el árbol.
- Si el árbol está vacío, se inserta en la raíz
- Si no está vacío, se inserta en el subárbol izquierdo si el valor es menor que el elemento en la raíz o en el derecho si es mayor



TAD Árbol binario de búsqueda

- **Nombre:** Árbol binario de búsqueda.
- **Valores posibles:** *vacío, otros*.
- **Operaciones:**
  - `add`
  - `remove`
  - `is_empty`
  - `in`
  - `initialize`
- **Uso de otros tipos:** tipo base, arrays / punteros (según implementación)

TAD Árbol binario de búsqueda

```
interface
  type
    tBinarySearchTree = ^tnode;
    tnode = RECORD
      info : integer;
      hi, hd : tBinarySearchTree;
    end;

  // Basic methods
  procedure initialize(var a:tBinarySearchTree);
  function is_empty(a:tBinarySearchTree): boolean;
  procedure add(var a:tBinarySearchTree; clave: integer);
  function in_tree(a: tBinarySearchTree; clave: integer): boolean;
  procedure remove(var a:tBinarySearchTree; clave: integer);

  // Traversal algorithms
  procedure preorder(a: tBinarySearchTree);
  procedure inorder(a: tBinarySearchTree);
  procedure postorder(a: tBinarySearchTree);

  // Other methods
  function node_count(a:tBinarySearchTree): integer;
  function node_arity(clave:integer): integer;
  function tree_arity(a:tBinarySearchTree): integer;
  function height(a:tBinarySearchTree): integer;
  function level(clave:integer): integer;

  function get_hi(a:tBinarySearchTree): tBinarySearchTree;
  function get_hd(a:tBinarySearchTree): tBinarySearchTree;
```

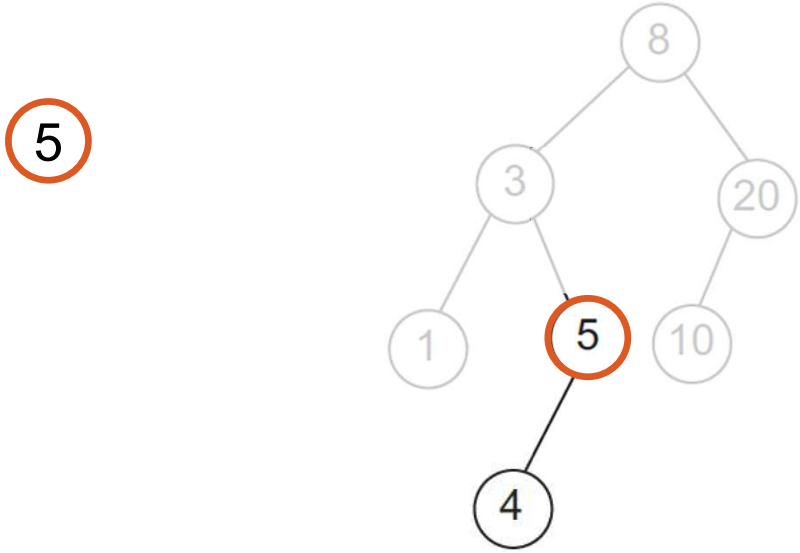
Búsqueda binaria en un BST

1. La clave buscada se compara con la clave del nodo raíz
2. Si las claves son iguales, la búsqueda se detiene
3. Si la clave buscada es mayor que la clave raíz, la búsqueda se reanuda en el subárbol derecho
4. Si la clave buscada es menor que la clave raíz, la búsqueda se reanuda con el subárbol izquierdo

- El proceso termina cuando se encuentra el valor buscado o se llega a un árbol vacío

Búsqueda binaria en un BST

```
function in_tree(a: tBinarySearchTree; clave: integer): boolean;
begin
  if a = NIL then in_tree := FALSE
  else if a^.info < clave then in_tree := in_tree(a^.hd, clave)
  else if a^.info > clave then in_tree := in_tree(a^.hi, clave)
  else in_tree := TRUE
end;
```

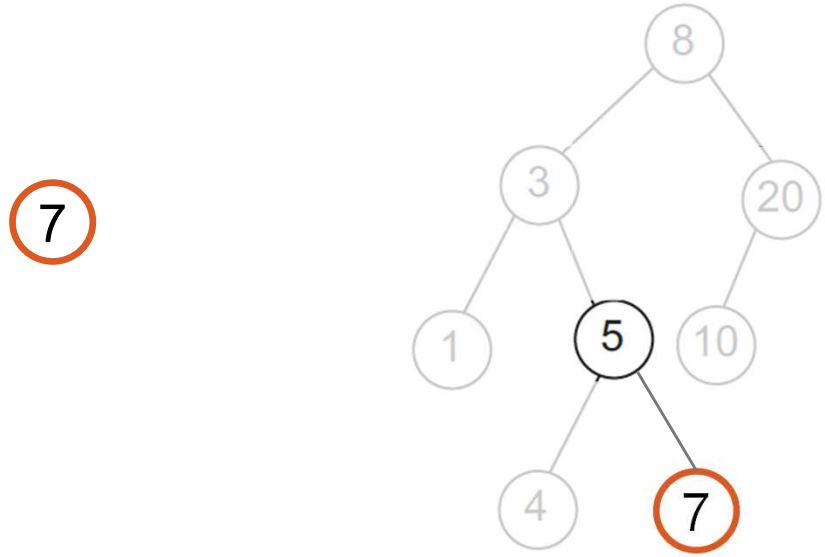


Inserción en un BST

- Una búsqueda binaria de una clave que no está en el árbol termina en un enlace nulo (subárbol vacío)
  - Al insertar sólo hay que sustituir ese enlace por un nuevo nodo que contenga la clave.
- El método de inserción recursivo utiliza una lógica similar a la empleada para la búsqueda recursiva:
  - Si el árbol está vacío, crear un nuevo nodo con la nueva clave
  - Si no, si la clave de búsqueda es menor que la clave de la raíz, insertar la clave en el subárbol izquierdo
  - Si es mayor, hacer lo mismo en el subárbol derecho

Inserción en un BST

```
procedure add(var a:tBinarySearchTree; clave: integer);
begin
  if a = NIL then begin
    new(a);
    a^.info := clave;
    a^.hi := NIL;
    a^.hd := NIL;
  end
  else if a^.info < clave then add(a^.hd, clave)
  else if a^.info > clave then add(a^.hi, clave)
end;
```



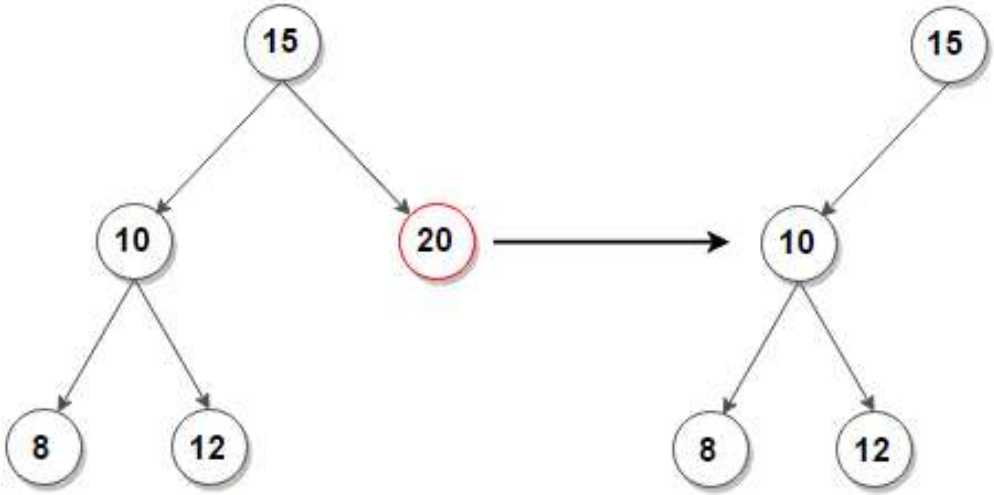
Borrado en un BST

- El borrado debe contemplar 4 casos:

  1. Si la clave a eliminar no está, no hacer nada
  2. Si la clave está en un nodo hoja, eliminar el nodo
  3. Si la clave está en un nodo con un solo hijo, eliminar el nodo y reemplazarlo por el subárbol hijo
  4. Si la clave a borrar está en un nodo con dos hijos, hay que buscar el valor sustituto, bien el mayor de los menores o el menor de los mayores, sustituirlo y posteriormente borrar el valor antiguo

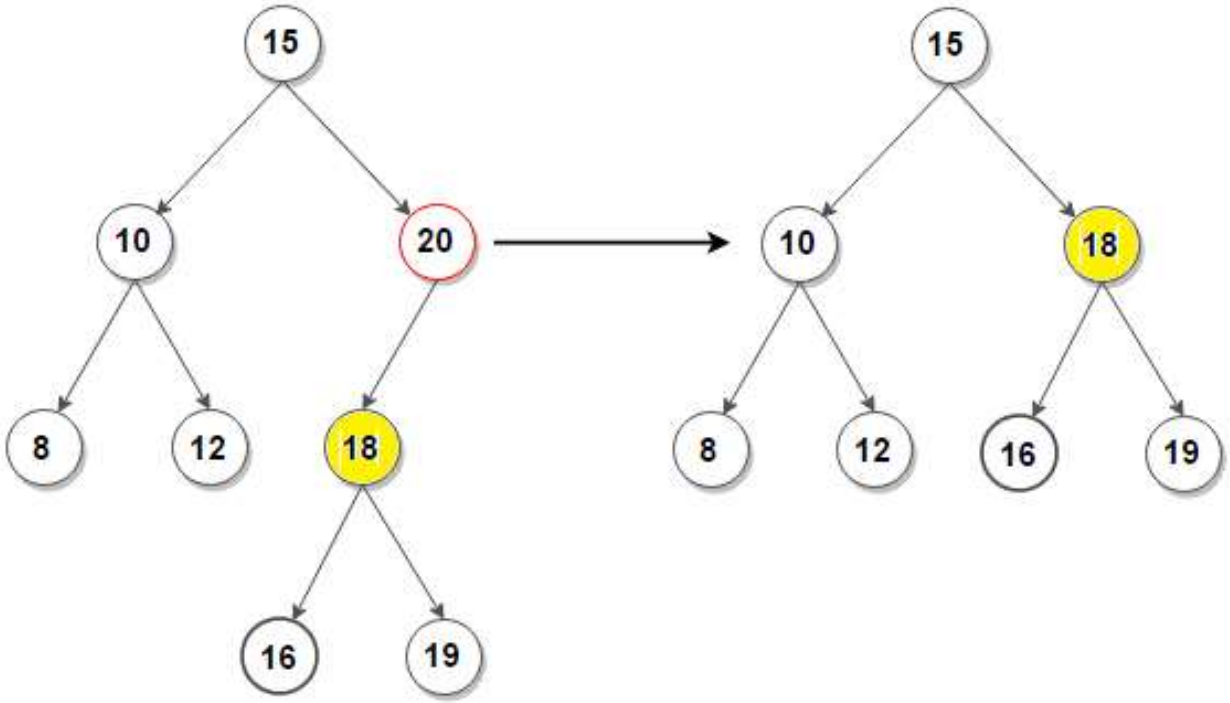
Borrado en un BST

[caso 2] La clave a borrar está en un nodo hoja



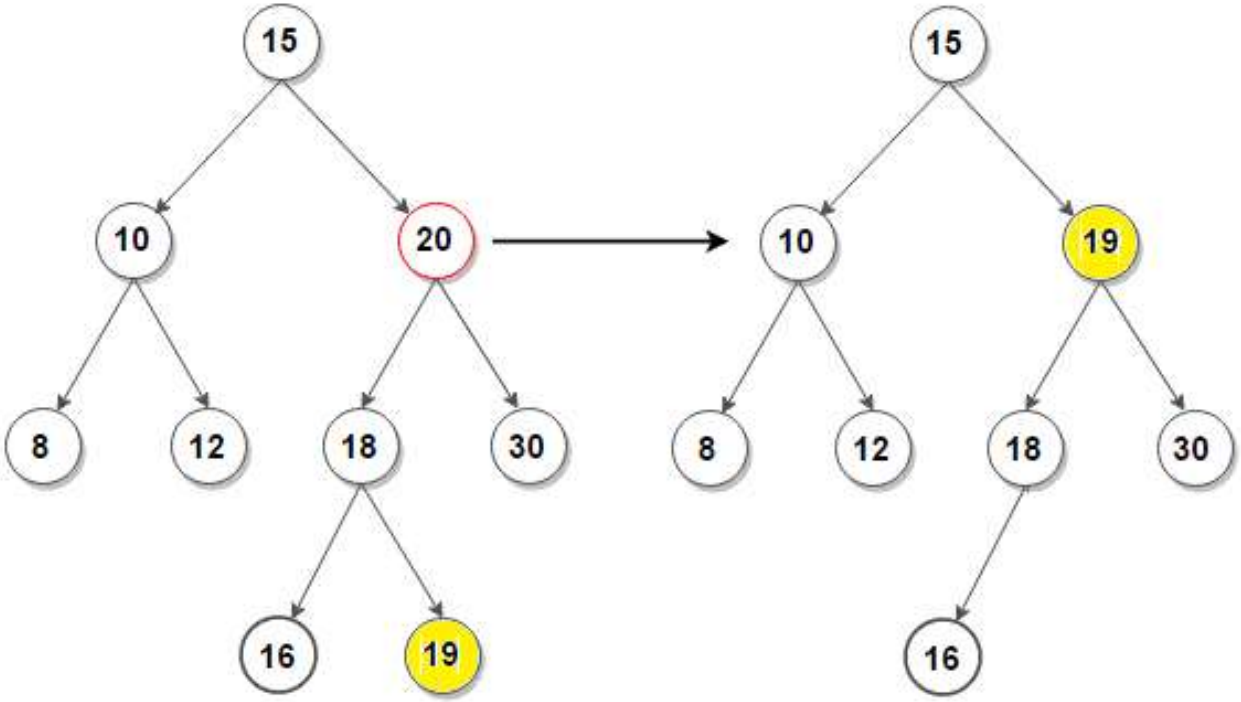
Borrado en un BST

[caso 3] La clave a borrar está en un nodo con un hijo



Borrado en un BST

[caso 4] La clave a borrar está en un nodo con dos hijos



Borrado en un BST

```
procedure remove(var a: tBinarySearchTree; x: integer);
var
  aux, ant: tBinarySearchTree;
begin
  if a <> NIL then
    if a^.info < x then remove(a^.hd, x)
    else if a^.info > x then remove(a^.hi, x)
    else
    begin
      aux := a;
      if a^.hi = NIL then a := a^.hd
      else if a^.hd = NIL then a := a^.hi
      else
        begin
          aux:= a^.hi;
          while aux^.hd <> NIL do
          begin
            ant:= aux;
            aux:= aux^.hd;
          end;
          if a^.hi = aux then a^.hi:= NIL else ant^.hd:= NIL;
          a^.info := aux^.info;
        end;
      dispose(aux)
    end
end;
```

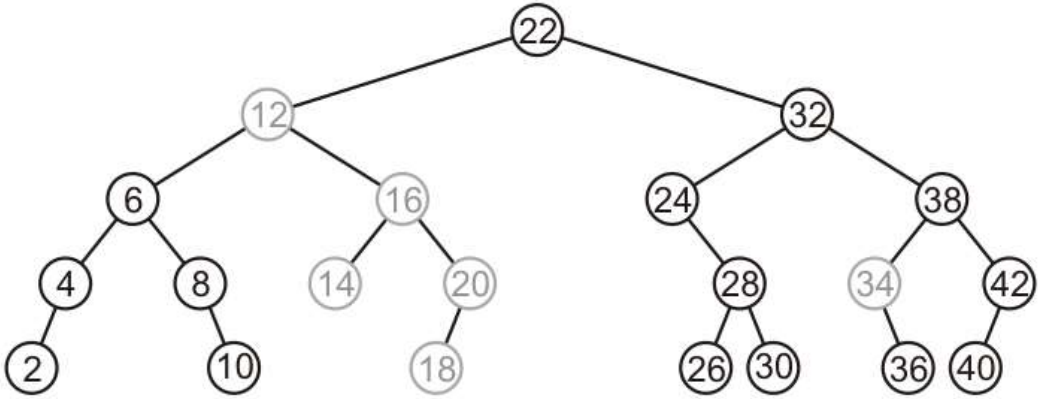
[caso 1]

[casos 2 y 3]

[caso 4]

Borrado “perezoso”

- Un árbol de búsqueda binario de borrado perezoso es un árbol de búsqueda binario en el que los objetos borrados simplemente se etiquetan como borrados
  - Los nodos permanecen físicamente en el árbol.
- Se puede invocar una función para limpiar (borrar) todos los nodos borrados a la vez cuando no sea crítico o pasar los nodos a otro árbol si hay pocos



Genericidad

Limitaciones del enfoque anterior

- Como en temas anteriores, hemos visto la aplicación de los árboles a un ejemplo sencillo: árboles de enteros
- Como en todas las estructuras de datos, es posible programar estructuras genéricas (con las limitaciones del lenguaje)
- Programar árboles genéricos comporta la implementación de:
  - `assign(var e: TElemento; e2: TElemento);`
  - `lessThan(e: TElemento; e2: TElemento): boolean;`
  - `equalTo(e: TElemento; e2: TElemento): boolean;`
  - …
- Muy importante en los árboles binarios de búsqueda la implementación de las operaciones de comparación pues se utilizan intensivamente

Árbol binario de búsqueda genérico

```
unit uBSTGenerico;

interface
  uses uTElemento;

  type
    tBSTGenerico = ^tnode;
    tnode = RECORD
      info : TElemento;
      hi,hd : tBSTGenerico;
    end;

  // Basic methods
  procedure initialize(var a:tBSTGenerico);
  function is_empty(a:tBSTGenerico): boolean;
  procedure add(var a:tBSTGenerico; clave: TElemento);
  function in_tree(a: tBSTGenerico; clave: TElemento): boolean;
  procedure remove(var a:tBSTGenerico; x: TElemento);

  // Traversal algorithms
  procedure preorder(a: tBSTGenerico);
  procedure inorder(a: tBSTGenerico);
  procedure postorder(a: tBSTGenerico);
```

Árbol binario de búsqueda genérico

```
procedure initialize(var a:tBSTGenerico);
begin
  a:= NIL;
end;

function is_empty(a:tBSTGenerico): boolean;
begin
  is_empty:= a=NIL;
end;

function in_tree(a: tBSTGenerico; clave: TElemento): boolean;
begin
  if a = NIL then in_tree := FALSE
  else if less_than(a^.info, clave) then in_tree := in_tree(a^.hd, clave)
  else if less_than(clave, a^.info) then in_tree := in_tree(a^.hi, clave)
  else in_tree := TRUE
end;

procedure add(var a:tBSTGenerico; clave: TElemento);
begin
  if a = NIL then begin
    new(a);
    assign(a^.info,clave);
    a^.hi := NIL;
    a^.hd := NIL;
  end
  else if less_than(a^.info, clave) then add(a^.hd, clave)
  else if less_than(clave, a^.info) then add(a^.hi, clave)
end;
```

Árbol binario de búsqueda genérico

```
procedure preorder(a: tBSTGenerico);
begin
  if (a <> NIL) then begin
    visit(a^.info);
    preorder(a^.hi);
    preorder(a^.hd)
  end
end;

procedure inorder(a: tBSTGenerico);
begin
  if (a <> NIL) then begin
    inorder(a^.hi);
    visit(a^.info);
    inorder(a^.hd)
  end
end;

procedure postorder(a: tBSTGenerico);
begin
  if (a <> NIL) then begin
    postorder(a^.hi);
    postorder(a^.hd);
    visit(a^.info);
  end
end;
```

preorden: **R-I-D**

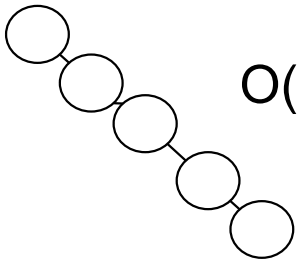
in-orden: **I-R-D**

post-orden: **I-D-R**

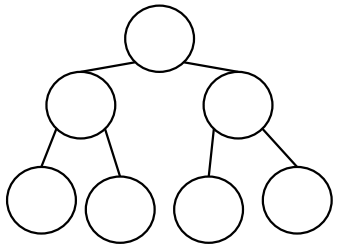
Árboles AVL

Eficiencia de las búsquedas

- En los árboles binarios de búsqueda se busca la mayor eficiencia posible, si bien esta operación depende del orden de inserción de los elementos



O(n)

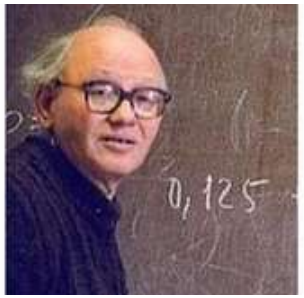


O(log n)

- La búsqueda es óptima en un árbol perfectamente equilibrado.
- Un BST no va a estar (casi nunca) perfectamente equilibrado, si bien también es difícil obtener un árbol muy degenerado.

Árboles AVL

- **Árbol equilibrado**: la altura del subárbol izquierdo y del subárbol derecho o son iguales o se diferencian en una unidad
- **AVL**: tipo especial de árbol binario de búsqueda "autobalanceable"
  - Ideado por los matemáticos soviéticos Georgi **A**delson-**V**elski y Eugenii **L**andis en 1962



*Georgi **A**delson-**V**elski y Eugenii **L**andis*

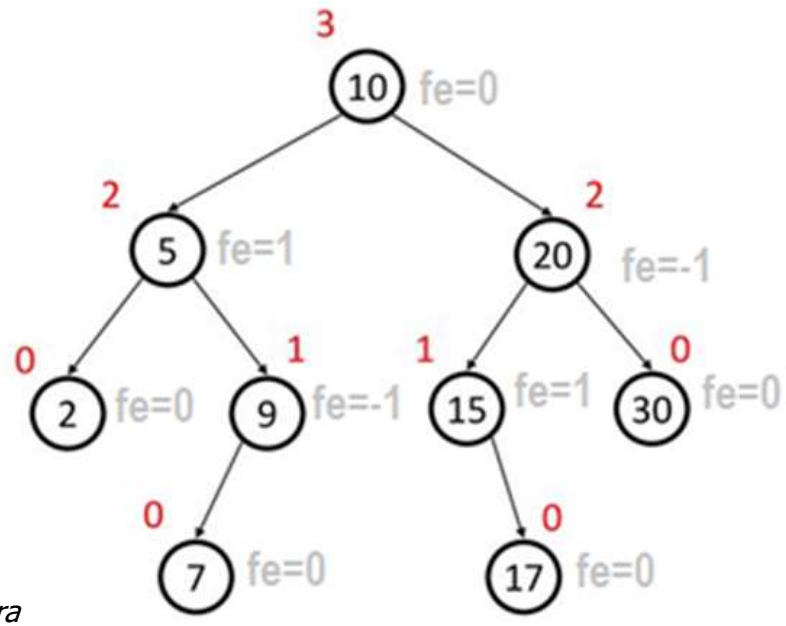
Árboles AVL

- La idea central es mantener balanceado el árbol
  - No solo en el momento de su construcción
  - Tampoco “periódicamente” invocando una operación “balancear”
  - Hacerlo tras cada inserción
- La operación “buscar” es igual de eficiente que en un BST
- El borrado y la inserción se debe actualizar la información de altura de los nodos y comprobar el equilibrio final: en caso necesario reequilibrar

Factor de equilibrio

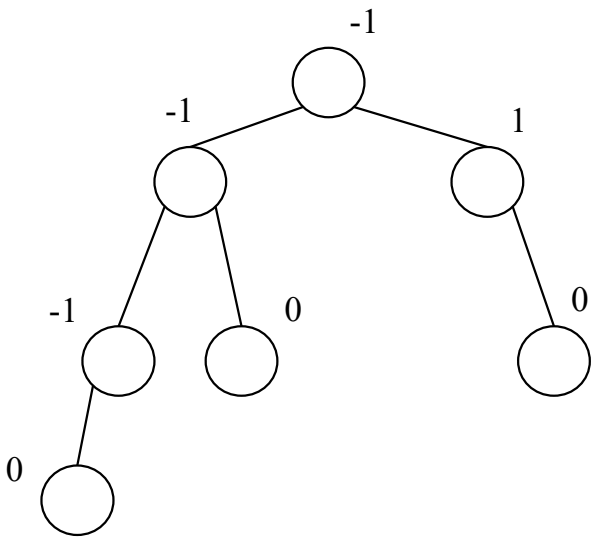
- Además de la propiedad de **orden**, los árboles balanceados mantienen una propiedad de **balanceo** que consiste en mantener un factor de equilibrio entre unos límites
- **Factor de equilibrio** de un nodo: altura de su subárbol derecho menos altura de su subárbol izquierdo
  - Este factor debe estar entre –1 y 1
  - Suele almacenarse en cada nodo
  - La altura del árbol vacío es -1

$$fe(nodo_i) = altura(hd)-altura(hi)$$

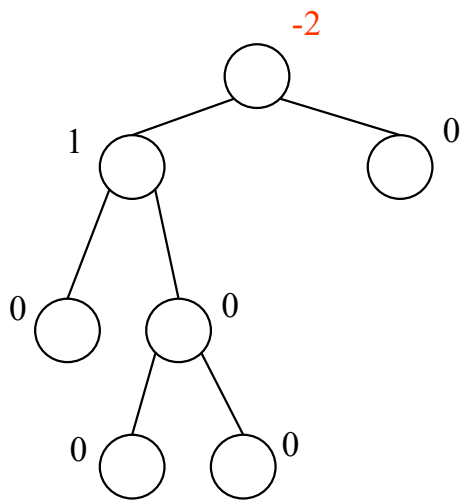


En rojo: altura
En gris: factor de equilibrio

¿Están en equilibrio?

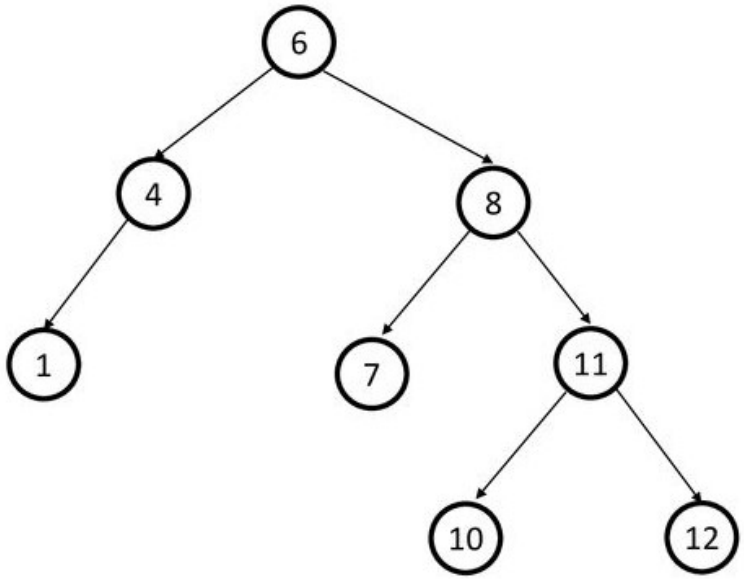


OK

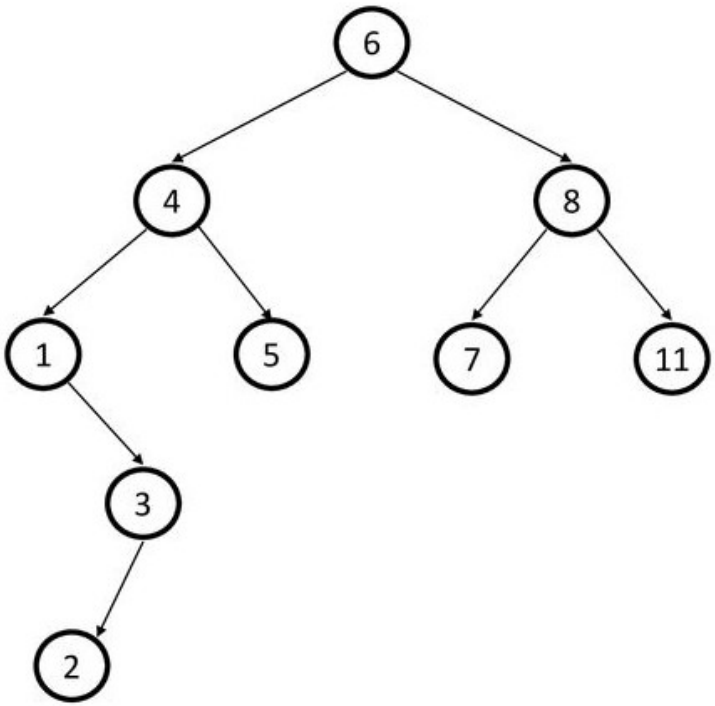


No OK

¿Están en equilibrio?



OK



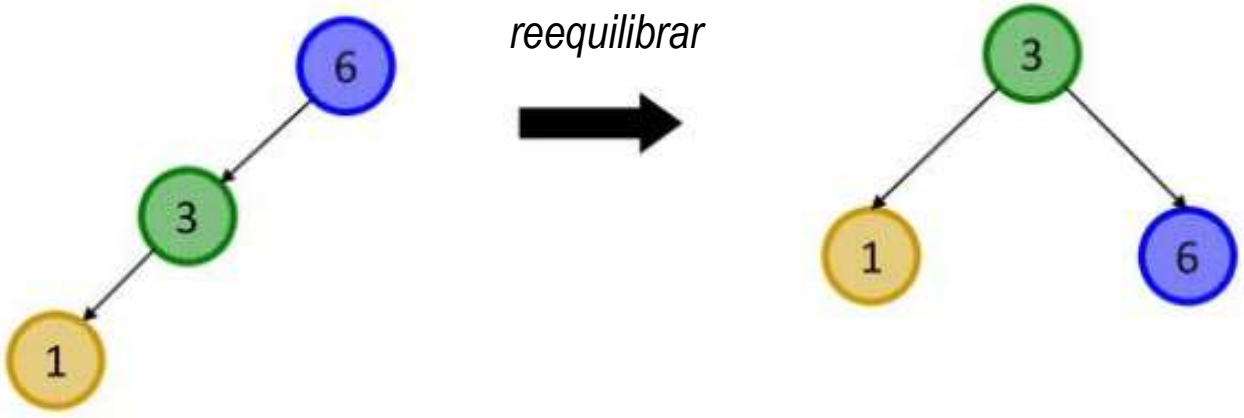
No OK

El problema del “desequilibrio”

- Una vez insertado el elemento se debe tener en cuenta si la inserción produjo un **desequilibrio**
  - Si la diferencia en altura de los hijos de un nodo es cero, una inserción nunca producirá desequilibrios
  - Si la diferencia en alturas es 1, insertando en el subárbol de mayor altura podría producirse desequilibrio
- El **reequilibrado** se realiza de abajo hacia arriba en los nodos en que se produce el desequilibrio.

Ejemplo preliminar

- Sobre un árbol vacío, insertemos 6, 3 y 1



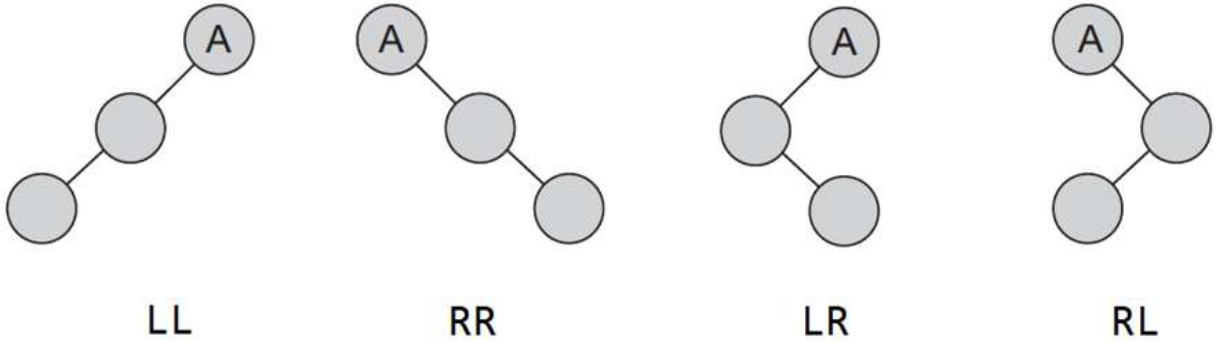
1. Mover el hijo del nodo en desequilibrio a la posición del padre
2. El padre se convertirá en el “otro hijo” del nodo

Rotaciones

- Cuando se inserta o elimina un nodo que rompe el equilibrio del árbol AVL entonces es necesaria una rotación
- Un estado no equilibrado es aquél en que algún subárbol tiene un factor de equilibrio mayor que 2 en valor absoluto
- Las rotaciones se llevan a cabo a partir de situaciones de desequilibrio y desencadenan acciones de corrección:
  - Hay 4 casos a tener en cuenta: LL, RR, LR, RL
  - Rotaciones: simple a izquierda, simple a derecha o doble

Situaciones de desequilibrio

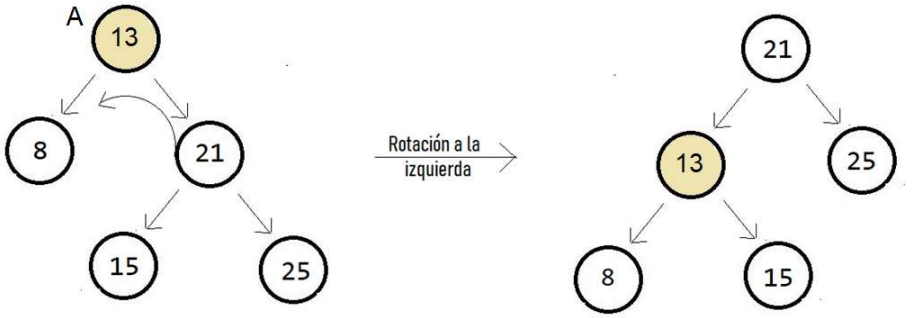
- A la vista de un nodo desequilibrado, se presentan 4 casos según dónde se haya hecho la inserción: LL, RR, LR, RL



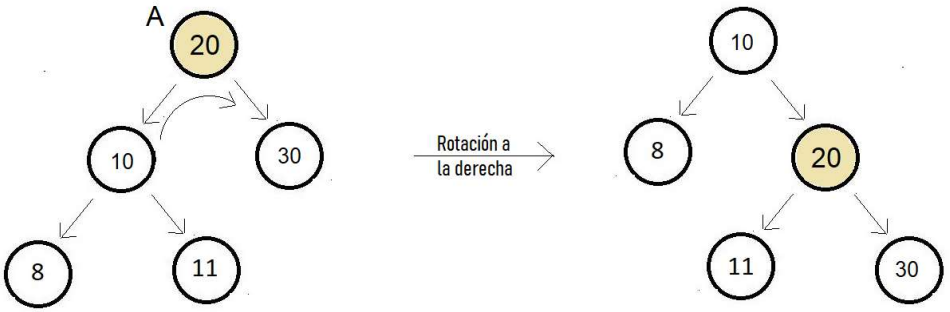
- Para reequilibrar:
  - LL – Rotar una vez a la derecha respecto al nodo en desequilibrio
  - RR – Rotar una vez a la izquierda respecto al nodo en desequilibrio
  - LR – Rotar una vez a la izquierda y luego otra a la derecha
  - RL – Rotar una vez a la derecha y luego otra a la izquierda

Rotaciones

- Hay dos rotaciones básicas: hacia la izquierda y hacia la derecha.
- La rotación simple implica a dos nodos, el nodo crítico A (nodo con |fe| = 2) y el descendiente izquierdo o derecho según el caso.
- RSI:

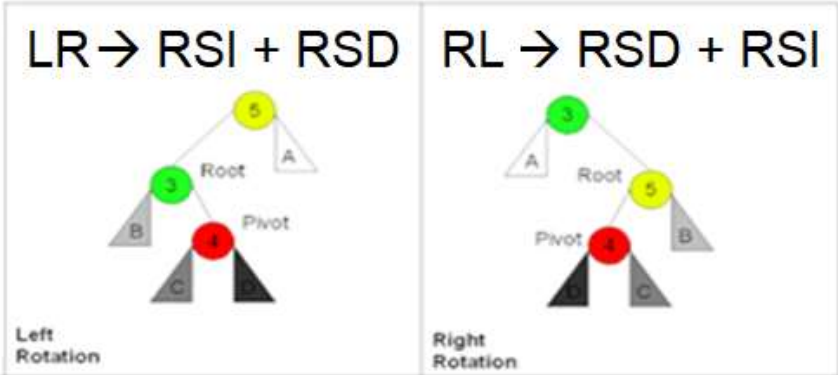


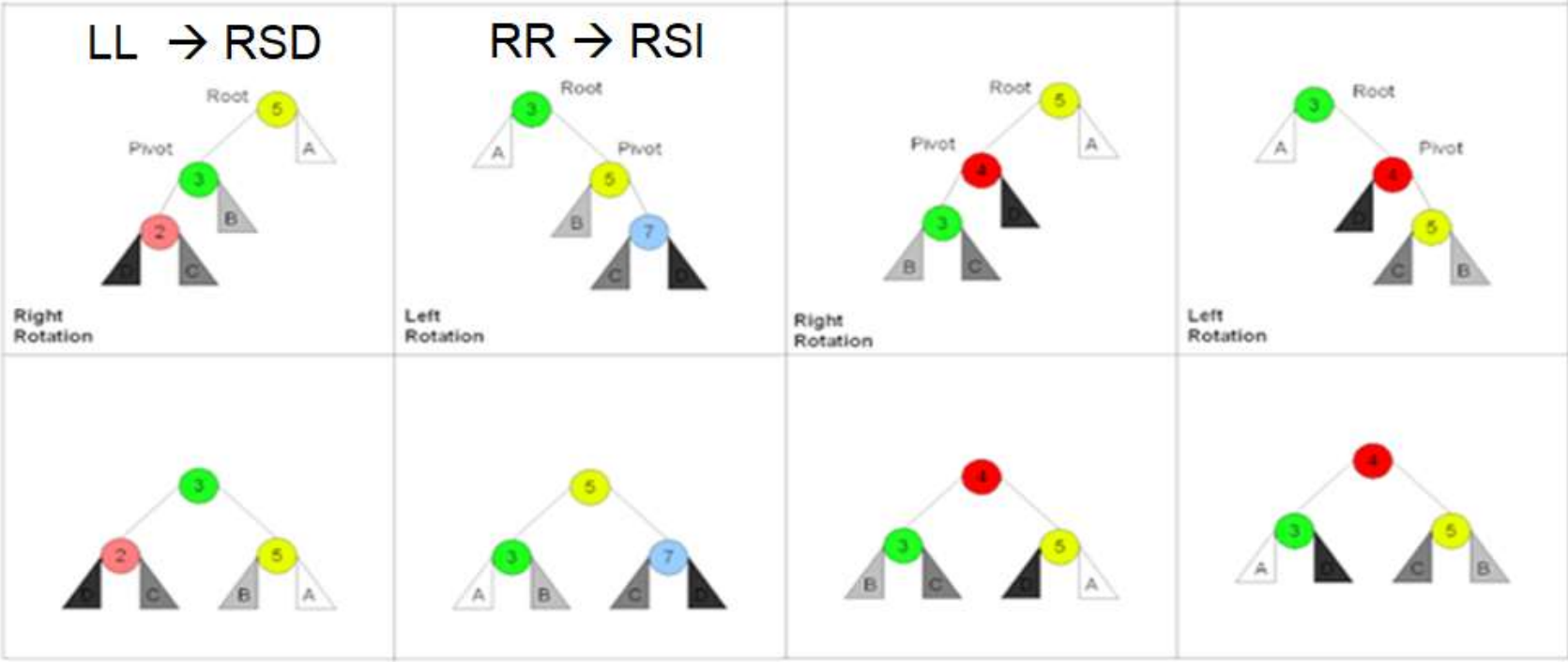
- RSD:



Algoritmo de rotaciones en AVL

si (altura hi – altura hd = 2) entonces
  si (altura hi de hi >= altura hd de hi) entonces RSD (LL)
  si no RD: RSI+RSD (LR)
si (altura hd – altura hi = 2) entonces
  si (altura hd de hd >= altura hi de hd) entonces RSI (RR)
  si no RD: RSD+RSI (RL)



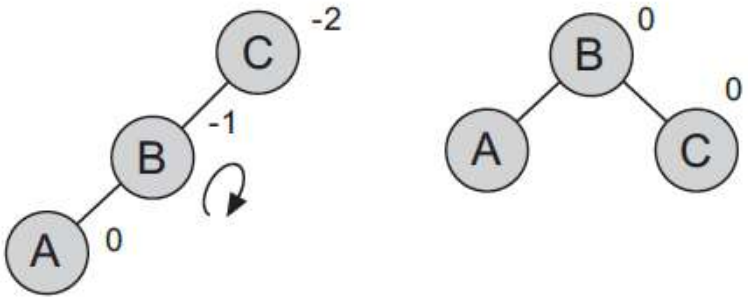


Rotación simple derecha (RSD)

- Detección situación LL:

si (altura hi - altura hd = 2) entonces
si (altura hi de hi >= altura hd de hi) entonces RSD

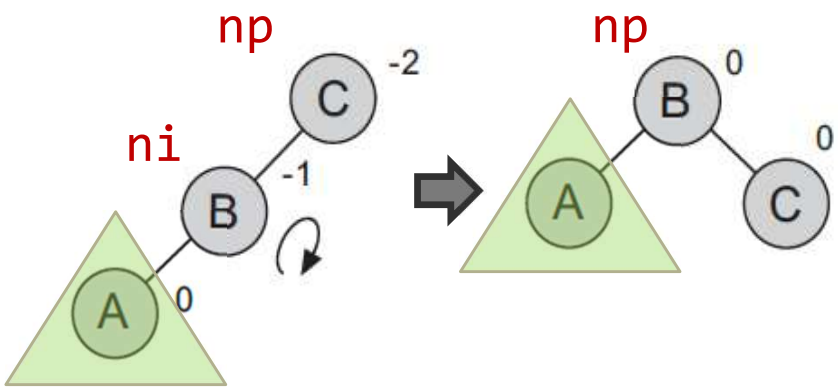
- Resolución: RSD



LL → RSD

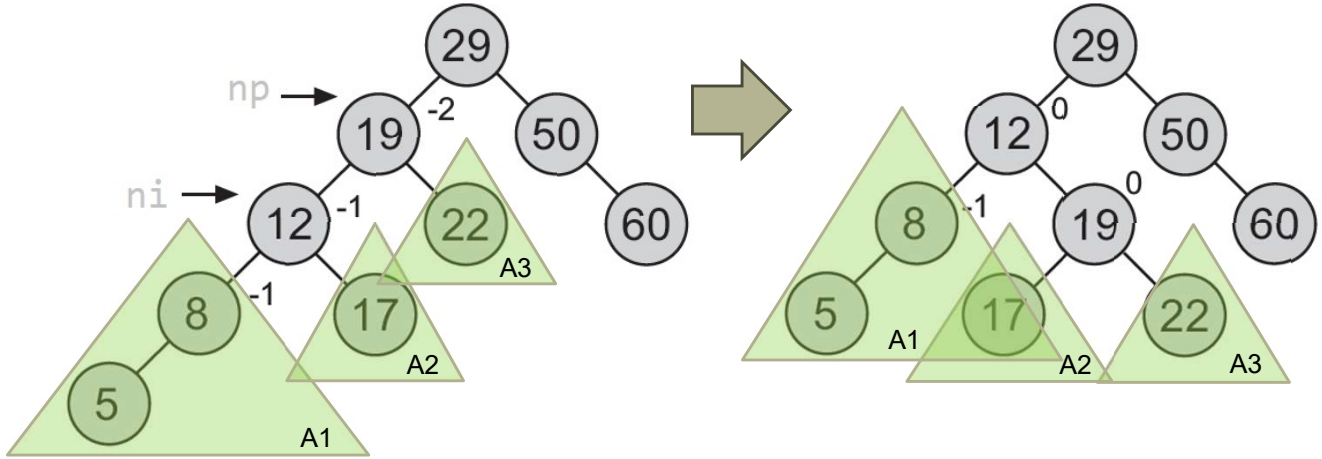
- Si llamamos `np` a la referencia al nodo problema y `ni` a la referencia al nodo de su rama izquierda, el código de resolución RSD sería:

```
np^.hi := ni^.hd;
ni^.hd := np;
np := ni;
```



LL → RSD (ejemplo)

- Ejemplo rotación simple RSD:



si (altura hi - altura hd = 2) entonces
si (altura hi de hi >= altura hd de hi) entonces RSD

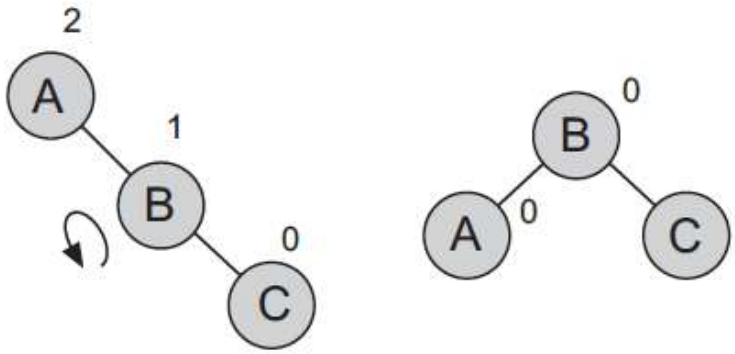
```
np^.hi := ni^.hd;
ni^.hd := np;
np := ni;
```

Rotación simple izquierda (RSI)

- Detección situación RR:

si (altura hd - altura hi = 2) entonces
  si (altura hd de hd >= altura hi de hd) entonces RSI

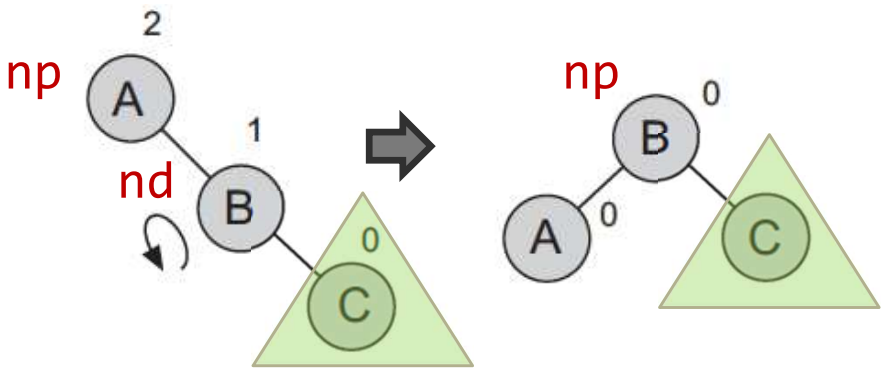
- Resolución: RSI



RR → RSI

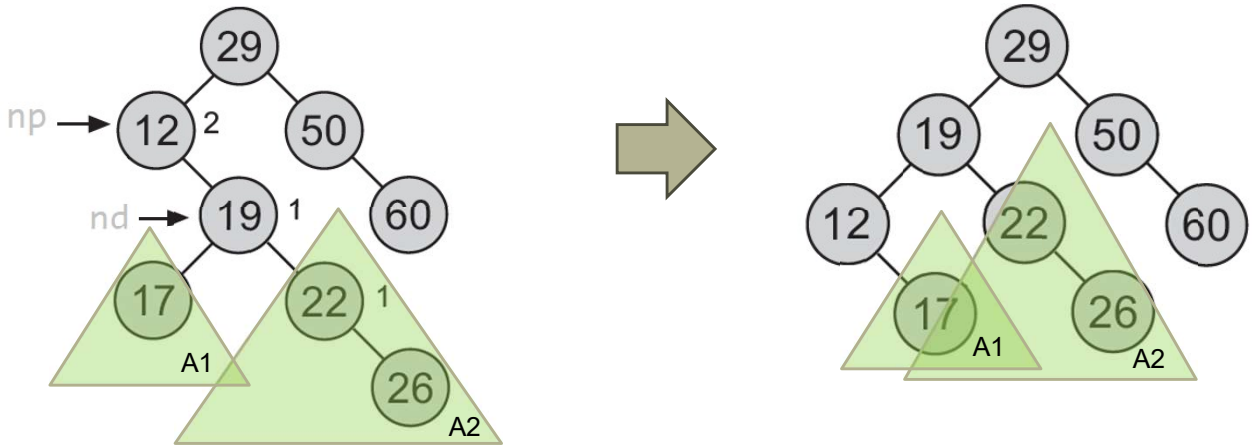
- Si llamamos np la referencia al nodo problema y nd la referencia al nodo de su rama derecha, el código de resolución RSI sería:

```
np^.hd := nd^.hi;
nd^.hi := np;
np := nd;
```



RR → RSI

- Ejemplo rotación simple RSI:

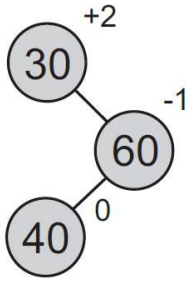


si (altura hd - altura hi = 2) entonces
  si (altura hd de hd >= altura hi de hd) entonces RSI

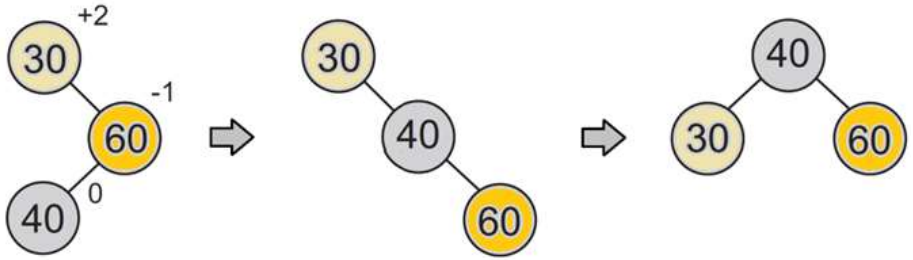
```
np^.hd := nd^.hi;
nd^.hi := np;
np := nd;
```

Rotaciones dobles

- La rotación simple no resuelve todos los casos de violación del criterio de equilibrio. Ejemplo:



- En casos como este, es necesaria una rotación doble para reequilibrar el árbol, que obliga a mover los enlaces de 3 nodos: el padre, el hijo (hd o hi) y el hijo del hijo por la rama contraria (hi o hd)



Rotación doble RDI

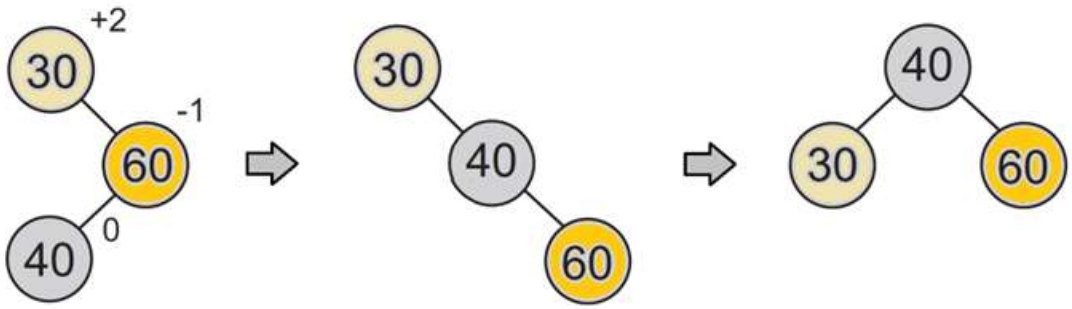
- En la rotación doble están implicados tres nodos, el nodo crítico A, su nodo hijo izquierdo y el hijo derecho de éste; o bien el caso simétrico, A, su hijo derecho y el hijo izquierdo de éste.
- Estas rotaciones resuelven las situaciones LR y RL
- RL → La rotación doble RL implica dos rotaciones:
  1. Una rotación simple derecha (RSD) sobre el hijo derecho del nodo crítico
  2. Una rotación simple izquierda (RSI) sobre el nodo crítico
- LR → La rotación doble LR implica dos rotaciones:
  1. Una rotación simple izquierda (RSI) sobre el hijo izquierdo del nodo crítico
  2. Una rotación simple derecha (RSD) sobre el nodo crítico

RL → RSD + RSI

- Detección situación RL:

si (altura hd – altura hi = 2) entonces
  si (altura hd de hd >= altura hi de hd) entonces RSI
  si no RD: RSD+RSI

- Resolución: RSD + RSI



RL → RSD + RSI

- Detección situación RL:

  si (altura hd – altura hi = 2) entonces
  si (altura hd de hd >= altura hi de hd) entonces RSI
  si no RD: RSD+RSI

- Resolución: RSD + RSI
- Si llamamos np a la referencia al nodo problema, n1 a la referencia a su hijo derecho y n2 al hijo izquierdo de n1, el código de resolución sería:

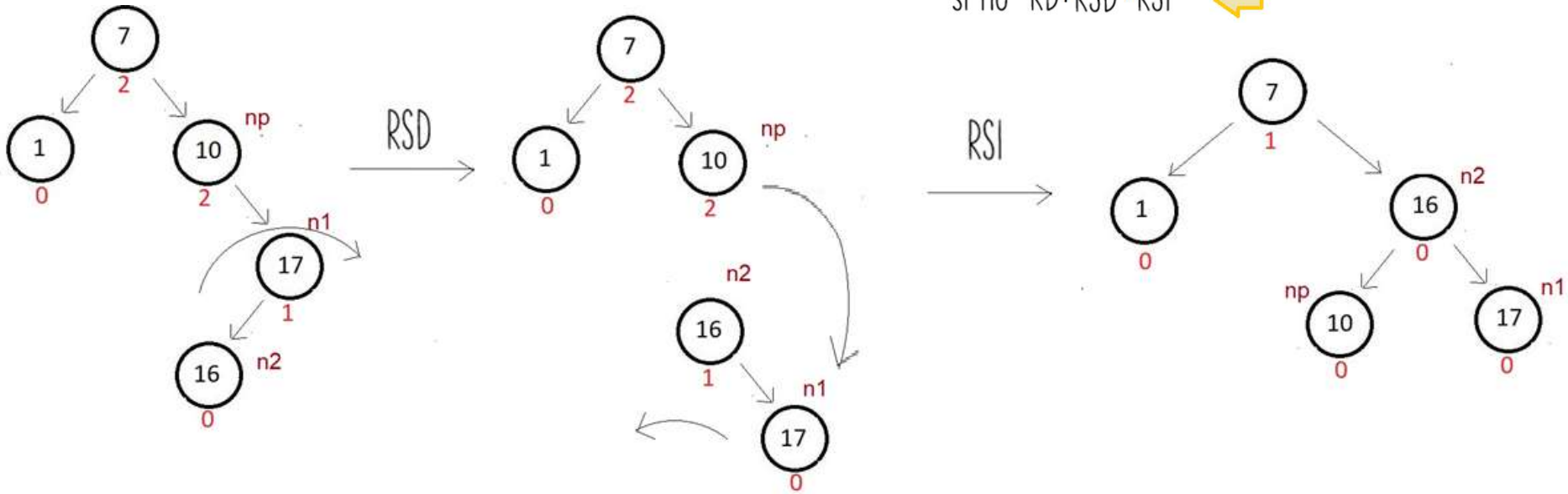
```
n1^.hi := n2^.hd;
n2^.hd := n1;
np^.hd := n2^.hi;
n2^.hi := np;
np := n2 ;
```

RSD sobre n1

RSI sobre np

Rotación RL (ejemplo)

si (altura hd – altura hi = 2) entonces
  si (altura hd de hd >= altura hi de hd) entonces RSI
  si no RD: RSD+RSI




```
n1^.hi := n2^.hd;
n2^.hd := n1;
```
RSD sobre n1

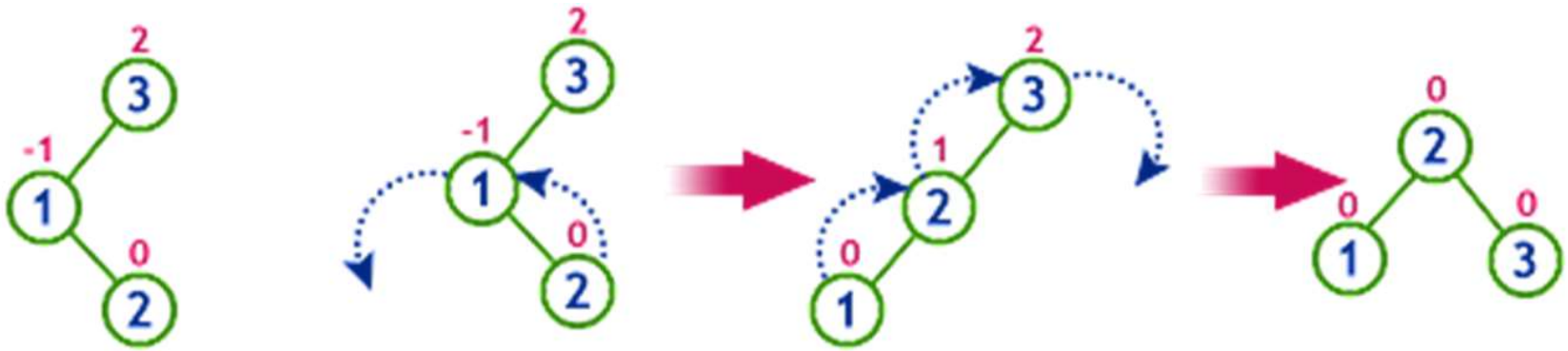
```
np^.hd := n2^.hi;
n2^.hi := np;
np := n2 ;
```
RSI sobre np

LR → RSI + RSD

- Detección situación LR:

si (altura hi - altura hd = 2) entonces
  si (altura hi de hi >= altura hd de hi) entonces RSD
  si no RD: RSI+RSD

- Resolución: RSI + RSD



LR → RSI + RSD

- Detección situación LR:

  si (altura hi - altura hd = 2) entonces
  si (altura hi de hi >= altura hd de hi) entonces RSD
  si no RD: RSI + RSD

- Resolución: RSI + RSD
- Si llamamos np a la referencia al nodo problema, n1 a la referencia a su hijo izquierdo y n2 al hijo derecho de n1, el código de resolución sería:

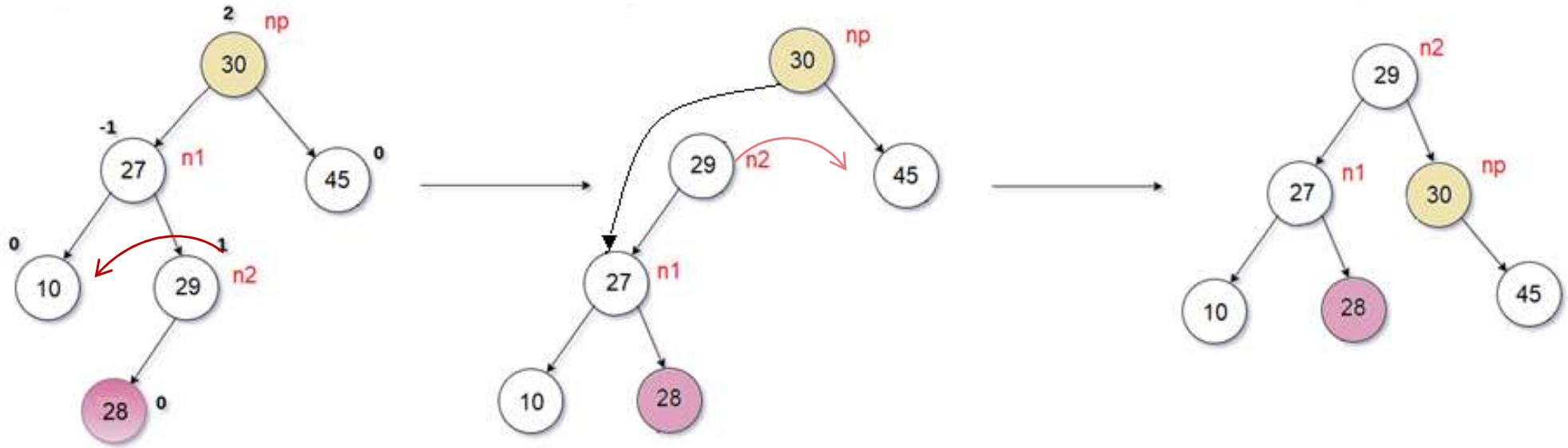
```
n1^.hd := n2^.hi;
n2^.hi := n1;
np^.hi := n2^.hd;
n2^.hd := np;
np := n2;
```

RSI sobre n1

RSD sobre np

Rotación LR (ejemplo)

si (altura hi - altura hd = 2) entonces
si (altura hi de hi >= altura hd de hi) entonces RSD
si no RD:RSI+RSD




```
n1^.hd := n2^.hi;
n2^.hi := n1;
```
} RSI sobre **n1**

```
np^.hi := n2^.hd;
n2^.hd := np;
np := n2;
```
} RSD sobre **np**

Rotaciones: resumen

- En cuanto al proceso de inserción en AVL:
  1. Insertar igual que en un árbol binario de búsqueda
  2. Almacenar el nuevo nodo con factor de equilibrio 0
  3. Desandar el camino de búsqueda, verificando el equilibrio de los nodos, y re-equilibrando si es necesario

- Dado un nodo, si la diferencia en valor absoluto entre las alturas de sus subárboles es 2, existe un desequilibrio
  - Para resolverlo hay que determinar cuál de los dos subárboles tiene mayor altura, lo que nos va a indicar qué rotación corresponde (izquierda o derecha)
  - Dentro de ese subárbol determinar nuevamente cuál de sus hijos tiene más altura (nos va a indicar si la rotación será simple o doble)

Bibliografía específica

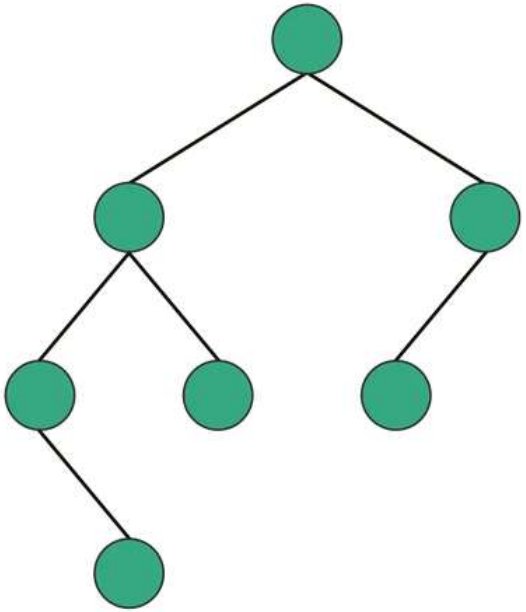
- Para más detalles sobre los procesos de rotación en árboles AVL, se recomienda el Tutorial de rotaciones de AVL de John Hargrove de la Univ. de Florida: https://www.cise.ufl.edu/~nemo/cop3530/AVL-Tree-Rotations.pdf

- También puede resultar de ayuda el capítulo Cap. 17 “Árboles de búsqueda equilibrados” del libro “Estructuras de datos en C++” de Luis Joyanes, Lucas Sánchez e Ignacio Zahonero, donde se explican muy claramente y con ejemplos los procesos de rotación.

Ejercicios propuestos

Ejercicio propuesto 1

Escribe un subprograma en Pascal que calcule el número de nodos de un árbol binario. Analiza la complejidad de tu solución.

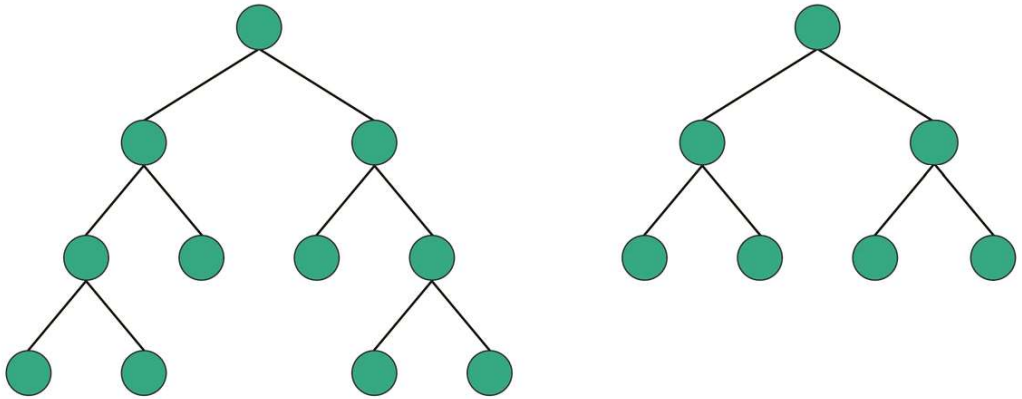


node_count = 7

Ejercicio propuesto 2

Se dice que un árbol binario está lleno cuando todos sus nodos tienen 2 hijos o ninguno (nodos hoja).

Escribe un subprograma en Pascal que determine si un árbol está lleno y calcula la complejidad de tu solución.



Ejercicio propuesto 2b

Hemos encontrado por Internet la siguiente implementación de la función que comprueba si un árbol está lleno. Presumimos que no es óptima y por tanto necesitamos analizar su complejidad en notación O grande. Estudia su complejidad y comenta los resultados.

```
function arbol_lleno(a:tBinarySearchTree): boolean;
begin
    if a = NIL then arbol_lleno:= true
    else if altura(a^.hi) <> altura(a^.hd) then arbol_lleno:= false
    else arbol_lleno:= arbol_lleno(a^.hi) and arbol_lleno(a^.hd)
end;
```

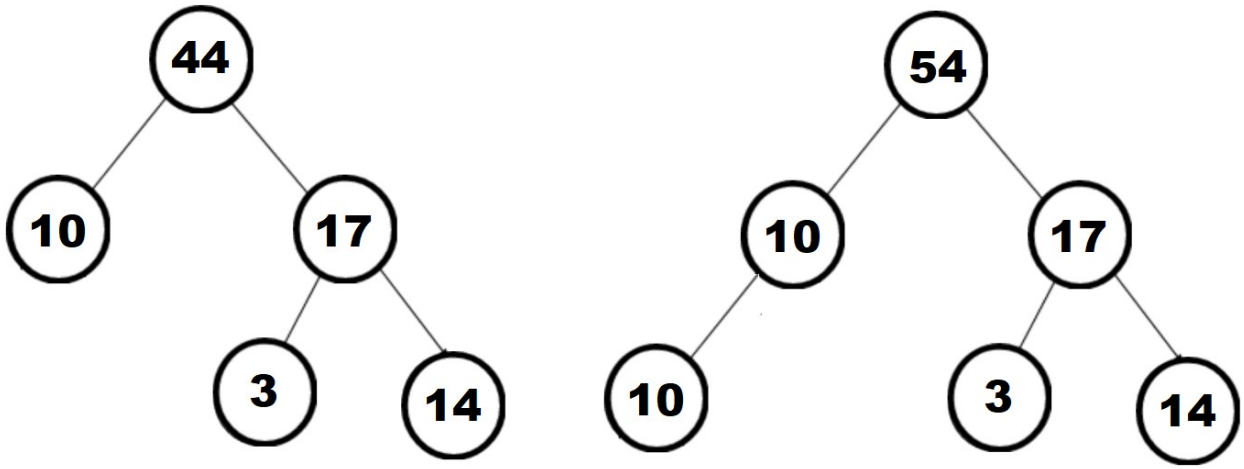
Ejercicio propuesto 3

Añadir al TAD Árbol Binario de Búsqueda una función que retorne el valor del nodo máximo. Analizar su complejidad y compararla con una función equivalente en un árbol binario general (no de búsqueda)

Ejercicio propuesto 4

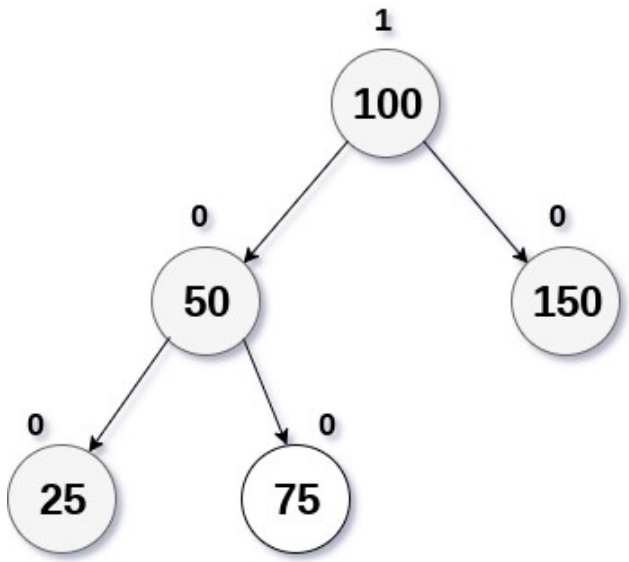
Escribe un método que verifique si un árbol binario es un árbol "suma". Los árboles "suma" tienen la característica de que todos sus nodos no hoja son iguales a la suma de sus nodos descendientes.

Por ejemplo, los siguientes son árboles "suma":



Ejercicio propuesto 5

Inserta el valor 12 en el siguiente árbol AVL y determina y lleva a cabo –si proceden- las operaciones de reequilibrio que sean necesarias.



Ejercicio propuesto 6 (reto)

- Escribe un programa que lea un texto de longitud indeterminada y que produzca como resultado un listado por pantalla de todas las palabras diferentes contenidas en el texto en orden alfabético junto con su frecuencia de aparición.
- El texto original se obtiene de un fichero de texto, para lo cual es recomendable descargar cualquier libro electrónico de la Biblioteca Virtual Miguel de Cervantes (https://www.cervantesvirtual.com/), libros que suelen tener cientos de miles de palabras.
- Emplea para tu solución un árbol binario de búsqueda, donde cada nodo contenga una palabra y su frecuencia.
- Compara la eficiencia de esta solución con la de una implementación que utilizase una lista ordenada para almacenar la información y reflexiona al respecto.